

Software Engineer || .NET Full Stack

Assignment Overview

Build a Smart Task Management System that demonstrates your ability to develop a modern, secure, and maintainable full-stack application using ASP.NET Core 9 or 10 and Angular.

The focus of this assignment is on code quality, security, maintainability, and best practices, rather than simply implementing CRUD operations.

Objectives

Your solution should demonstrate your ability to:

- Build a clean and scalable application
- Develop secure REST APIs
- Create a responsive Angular frontend
- Follow SOLID principles
- Apply design patterns where appropriate
- Document your solution clearly
- Integrate a simple AI-powered feature

Technology Stack

Backend

- ASP.NET Core 9 or 10
- Entity Framework Core
- SQL Server
- JWT Authentication
- Swagger / OpenAPI

Frontend

- Angular 18+
- TypeScript
- Standalone Components

Functional Requirements

Authentication & Authorization

Implement:

- User Registration
- Login
- Logout
- JWT Authentication
- Refresh Tokens
- Role-Based Authorization

Roles

- Admin
- Project Manager
- Team Member

Each role should have appropriate permissions.

Project Management

Users should be able to:

- Create Project
- Update Project
- Delete Project
- View Project Details
- List Projects
- Search Projects
- Sort and Paginate Results

Task Management

Each project contains multiple tasks.

Implement:

- Create Task
- Update Task
- Delete Task
- Assign Task
- Update Status
- Set Priority
- Due Date
- Search Tasks
- Sort and Paginate Tasks

Task Status

- To Do

- In Progress
- Completed
- Cancelled

Priority

- Low
- Medium
- High
- Critical

Dashboard

Display basic statistics such as:

- Total Projects
- Total Tasks
- Tasks by Status
- Tasks by Priority
- Completed vs Pending Tasks
- Upcoming Due Tasks

Search, Sorting & Pagination

Support:

- Keyword Search
- Filtering
- Sorting
- Pagination

AI Feature

Create an API endpoint that improves task descriptions.

The AI should:

- Correct grammar
- Improve clarity
- Make descriptions more professional
- Expand short descriptions
- Produce actionable task descriptions

You may use **GitHub Models (Free)** or any free AI provider.

Architecture

Use **N-Layer Architecture** or **Clean Architecture**.

Security

Implement:

- JWT Authentication
- Refresh Tokens
- Password Hashing
- Role-Based Authorization
- HTTPS
- CORS
- Input Validation
- Basic Rate Limiting
- Secure Configuration

Backend Expectations

Include:

- Async/Await
- FluentValidation
- Swagger
- Global Exception Handling
- Logging (Serilog)
- Health Checks
- Consistent API Responses

Frontend Expectations

Implement:

- Login
- Dashboard
- Project Management
- Task Management
- Search
- Responsive UI

Follow Angular best practices including:

- Lazy Loading

- Route Guards
- HTTP Interceptors
- Reactive Forms

Documentation

Provide the following files:

README.md

Include:

- Project Overview
- Setup Instructions
- Technology Stack
- API Overview
- Folder Structure

PROMPTS.md

Describe:

- AI Prompt Design
- Prompt Structure
- Example Inputs and Outputs
- Validation Approach
- Safety Considerations

Evaluation Criteria

Your submission will be evaluated based on:

- Application Architecture
- Code Quality
- Security
- Backend Development
- Frontend Development
- Design Patterns
- Documentation
- Readability and Maintainability

Recommended Development Order

- Design the Architecture
- Implement Authentication

- Build Project Module
- Build Task Module
- Add Search, Sorting & Pagination
- Create Dashboard
- Integrate AI Feature
- Build Angular Frontend
- Complete Documentation

Deliverables

Submit:

- Complete Source Code managed in a GitHub repository
- GitHub repository link shared with the submission
- SQL Database Script or EF Core Migrations
- README.md
- PROMPTS.md
- API Collection (Postman or Swagger Export)

Final Goal

Deliver a production-ready **Smart Task Management System** that demonstrates strong software engineering skills, secure API development, modern Angular development, and clear documentation.

Good engineering practices, maintainability, and code quality are more important than implementing every possible feature.